Лабораторная работа №4

Бессонов Андрей Максимович

Содержание

1 Цель работы **5**

2 Задание **6**

3 Теоретическое введение **7**
3.1 Рабочий процесс Gitflow 7
3.2 Семантическое версионирование 8
3.3 Общепринятые коммиты (Conventional Commits) 8

4 Выполнение лабораторной работы **9**
4.1 Установка необходимого ПО 9
4.2 Создание локального репозитория и первый коммит 10
4.3 Инициализация git-flow 13
4.4 Создание первого релиза (v1.0.0) 15
4.5 Создание релиза на GitHub 16
4.6 Создание релиза v1.2.3 17
4.7 Выполнение задания 18

5 Выводы **22**

Список литературы **23**

Список иллюстраций

4.1 Рис.1 9
4.2 Рис.2 11
4.3 Рис.3 12
4.4 Рис.4 14
4.5 Рис.5 15
4.6 Рис.6 16
4.7 Рис.7 18
4.8 Рис.8 19
4.9 Рис.9 20
4.10 Рис.10 20
4.11 Рис.11 21

Список таблиц

1 Цель работы

Получение навыков правильной работы с репозиториями git, освоение модели ветвления Gitflow, семантического версионирования и стандартизации коммитов.

2 Задание

Выполнить работу для тестового репозитория. Преобразовать рабочий репозиторий в репозиторий с git-flow и conventional commits.

3 Теоретическое введение

3.1 Рабочий процесс Gitflow

- Gitflow Workflow — это строгая модель ветвления, ориентированная на управление релизами. Основные положения:
- master — хранит официальную историю релизов. Каждый коммит на master соответствует релизу и должен быть помечен тегом версии.
- develop — ветка разработки, в которой интегрируются все новые функции. Служит базой для создания веток функций и релизов.
- feature — ветки для разработки отдельных функций. Создаются от develop и после завершения сливаются обратно в develop.
- release — ветки подготовки релиза. Создаются от develop, когда набрано достаточно функций для выпуска. Допускается только отладка и документация. После завершения сливаются в master и develop.
- hotfix — ветки для срочных исправлений в работающем релизе. Создаются от master и после завершения сливаются в master и develop.
- Для работы с Gitflow используется пакет git-flow. Инициализация выполняется командой git flow init. После этого создаётся ветка develop, и все новые функции разрабатываются в ветках feature.

3.2 Семантическое версионирование

Семантическое версионирование (SemVer) задаёт формат номера версии: MAJOR.MINOR.PATCH. - MAJOR увеличивается при несовместимых изменениях API. - MINOR — при добавлении новой функциональности с обратной совместимостью. - PATCH — при обратно совместимых исправлениях ошибок. - Номер версии фиксируется в файле package.json и в тегах репозитория.

3.3 Общепринятые коммиты (Conventional Commits)

Спецификация Conventional Commits регламентирует структуру сообщений коммитов, что позволяет автоматически определять следующий номер версии и генерировать журнал изменений. Основные типы коммитов: - feat: — новая функция (MINOR). - fix: — исправление ошибки (PATCH). - BREAKING CHANGE: — несовместимое изменение (MAJOR), может быть частью любого типа. - docs:, style:, refactor:, perf:, test:, chore: и др. — не влияют на версию, но используются для каталогизации изменений. - Для помощи в написании коммитов по стандарту используется утилита commitizen. Генерация журнала изменений выполняется с помощью standard-changelog.

4 Выполнение лабораторной работы

В ходе работы мы выполнили все поставленные задачи:

4.1 Установка необходимого ПО

На ОС Fedora были выполнены следующие команды для установки git-flow, Node.js и инструментов: - Установка git-flow из репозитория copr sudo dnf copr enable elegos/gitflow sudo dnf install gitflow - Установка Node.js и pnpm sudo dnf install nodejs pnpm - Настройка pnpm (добавление в PATH) pnpm setup source ~/.bashrc - Установка commitizen и standard-changelog глобально pnpm add -g commitizen standard-changelog

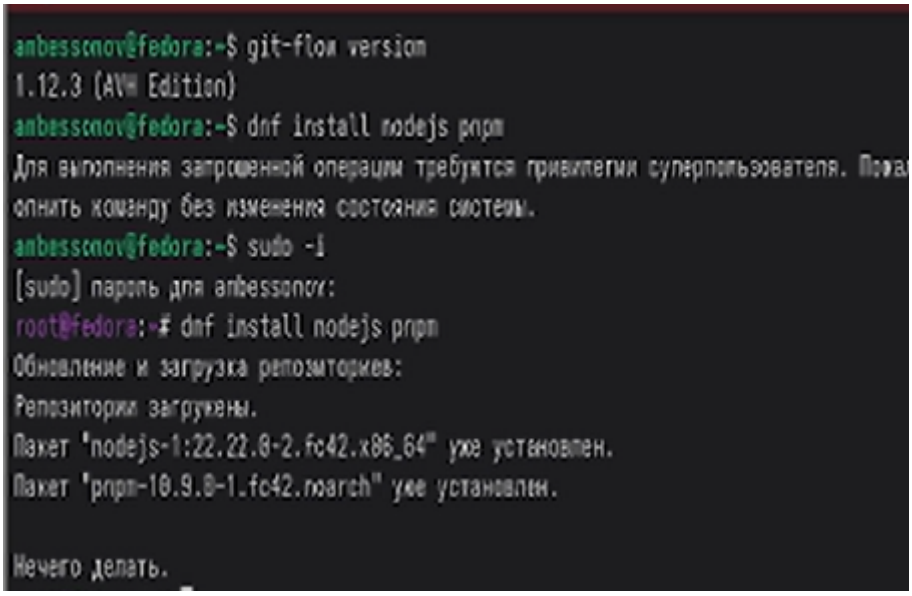

Рисунок 4.1: Рис.1

4.2 Создание локального репозитория и первый коммит

Был создан каталог для нового репозитория и инициализирован Git:

- mkdir ~/git-extended
- cd ~/git-extended
- git init
- echo «# git-extended» > README.md
- git add README.md
- git commit -m «first commit»
- git branch -M master
- git remote add origin git@github.com:ambessonov980/git-extended.git
- git push -u origin master

Настройка package.json и commitizen Для использования commitizen был инициализирован пакет Node.js:

pnpm init

После завершения wizard'а в файл package.json была добавлена секция config для настройки commitizen: json «config»: { «commitizen»: { «path»: «cz-conventional-changelog» } }

Затем выполнено добавление файлов и создание первого коммита с помощью git cz: git add . git cz Был выбран тип chore, сообщение init project.

```
ambassonov@fedora:~$ pnpm setup
source ~/.bashrc
No changes to the environment were made. Everything is already up to date.
ambassonov@fedora:~$ pnpm add -g commitizen standard-changelog
 WARN  8 deprecated subdependencies found: glob@7.2.3, har-validator@5.1.5, inflight@1.0.6, lodash.get@
Already up to date
Progress: resolved 392, reused 392, downloaded 0, added 0, done
 WARN  Issues with peer dependencies found
.
└─┬ gh 2.8.9
  └─┬ marked-terminal 3.2.0
    └── X unmet peer marked@"^0.4.0 || ^0.5.0 || ^0.6.0": found 0.7.0
Done in 4.4s using pnpm v10.9.0
ambassonov@fedora:~$ mkdir ~/git-extended
cd ~/git-extended
git init
echo "# git-extended" > README.md
git add README.md
git commit -m "first commit"
git branch -M master
git remote add origin git@github.com:ambassonov308/git-extended.git
git push -u origin master
mkdir: cannot create directory '/home/ambassonov/git-extended': Файл существует
Переинициализирован существующий репозиторий Git в /home/ambassonov/git-extended/.git/
Текущая ветка: master
Эта ветка соответствует «origin/master».

Неотслеживаемые файлы:
  (используйте «git add <файл>...», чтобы добавить в то, что будет включено» в коммит)
        package.json

индекс пуст, но есть неотслеживаемые файлы
(используйте «git add», чтобы проиндексировать их)
error: внешний репозиторий origin уже существует
branch 'master' set up to track 'origin/master'.
Everything up-to-date
ambassonov@fedora:~/git-extended$ pnpm init
```

Рисунок 4.2: Рис.2

```
ambessonov@fedora:~/git-extended$ git add .
git cz
cz-cli@4.3.1, cz-conventional-changelog@3.3.0

? Select the type of change that you're committing: chore:   Other changes that don't modify src or test files
? What is the scope of this change (e.g. component or file name): (press enter to skip)
? Write a short, imperative tense description of the change (max 93 chars):
 (12) init project
? Provide a longer description of the change: (press enter to skip)

? Are there any breaking changes? No
? Does this change affect any open issues? No
[master 4142d68] chore: init project
 1 file changed, 19 insertions(+)
 create mode 100644 package.json
ambessonov@fedora:~/git-extended$ git flow init

Which branch should be used for bringing forth production releases?
   - master
Branch name for production releases: [master]
Branch name for "next release" development: [develop]

How to name your supporting branch prefixes?
Feature branches? [feature/]
Bugfix branches? [bugfix/]
Release branches? [release/]
Hotfix branches? [hotfix/]
Support branches? [support/]
Version tag prefix? [] v
Hooks and filters directory? [/home/ambessonov/git-extended/.git/hooks]
ambessonov@fedora:~/git-extended$ git push --all
git branch --set-upstream-to=origin/develop develop
Перечисление объектов: 4, готово.
Подсчет объектов: 100% (4/4), готово.
При сжатии изменений используется до 16 потоков
Сжатие объектов: 100% (3/3), готово.
Запись объектов: 100% (3/3), 1.22 КиБ | 138.00 КиБ/с, готово.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To github.com:ambessonov000/git-extended.git
   b3583a0..4142d68  master -> master
 * [new branch]      develop -> develop
branch 'develop' set up to track 'origin/develop'.
```

Рисунок 4.3: Рис.3

4.3 Инициализация git-flow

В репозитории выполнена инициализация git-flow: - git flow init После инициализации произошло автоматическое переключение на ветку develop. Проверено: - git branch Затем ветка develop была отправлена на GitHub и настроена как вышестоящая: - git push –all - git branch –set-upstream-to=origin/develop develop

```
ambassonov@fedora:~/git-extended$ git push --all
git branch --set-upstream-to=origin/develop develop
Перечисление объектов: 4, готово.
Подсчет объектов: 100% (4/4), готово.
При сжатии изменений используется до 16 потоков
Сжатие объектов: 100% (3/3), готово.
Запись объектов: 100% (3/3), 1.22 КиБ | 138.00 КиБ/с, готово.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To github.com:ambassonov999/git-extended.git
   b3583a0..4142068  master -> master
 * [new branch]      develop -> develop
branch 'develop' set up to track 'origin/develop'.
ambassonov@fedora:~/git-extended$ git flow release start 1.0.0
standard-changelog --first-release
git add CHANGELOG.md
git commit -am 'chore(site): add changelog'
git flow release finish 1.0.0
Переключились на новую ветку «release/1.0.0»

Summary of actions:
- A new branch 'release/1.0.0' was created, based on 'develop'
- You are now on branch 'release/1.0.0'

Follow-up actions:
- Bump the version number now!
- Start committing last-minute fixes in preparing your release
- When done, run:

     git flow release finish '1.0.0'

[release/1.0.0 e6710a3] chore(site): add changelog
 1 file changed, 1 insertion(+)
 create mode 100644 CHANGELOG.md
Переключились на ветку «master»
Эта ветка соответствует «origin/master».
Merge made by the 'ort' strategy.
 CHANGELOG.md | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 CHANGELOG.md
Уже на «master»
Ваша ветка опережает «origin/master» на 2 коммита.
  (используйте «git push», чтобы опубликовать ваши локальные коммиты)
fatal: нет описания метки?
Fatal: Tagging failed. Please run finish again to retry.
```

Рисунок 4.4: Рис.4

4.4 Создание первого релиза (v1.0.0)

Начата релизная ветка для версии 1.0.0: - git flow release start 1.0.0 Сгенерирован начальный журнал изменений: - standard-changelog –first-release Файл CHANGELOG.md был добавлен и закоммичен: - git add CHANGELOG.md - git commit -am „chore(site): add changelog“ Завершён релиз (ветка вливается в master и develop, проставляется тег): - git flow release finish 1.0.0 После этого все изменения и теги отправлены на GitHub: - git push –all - git push –tags

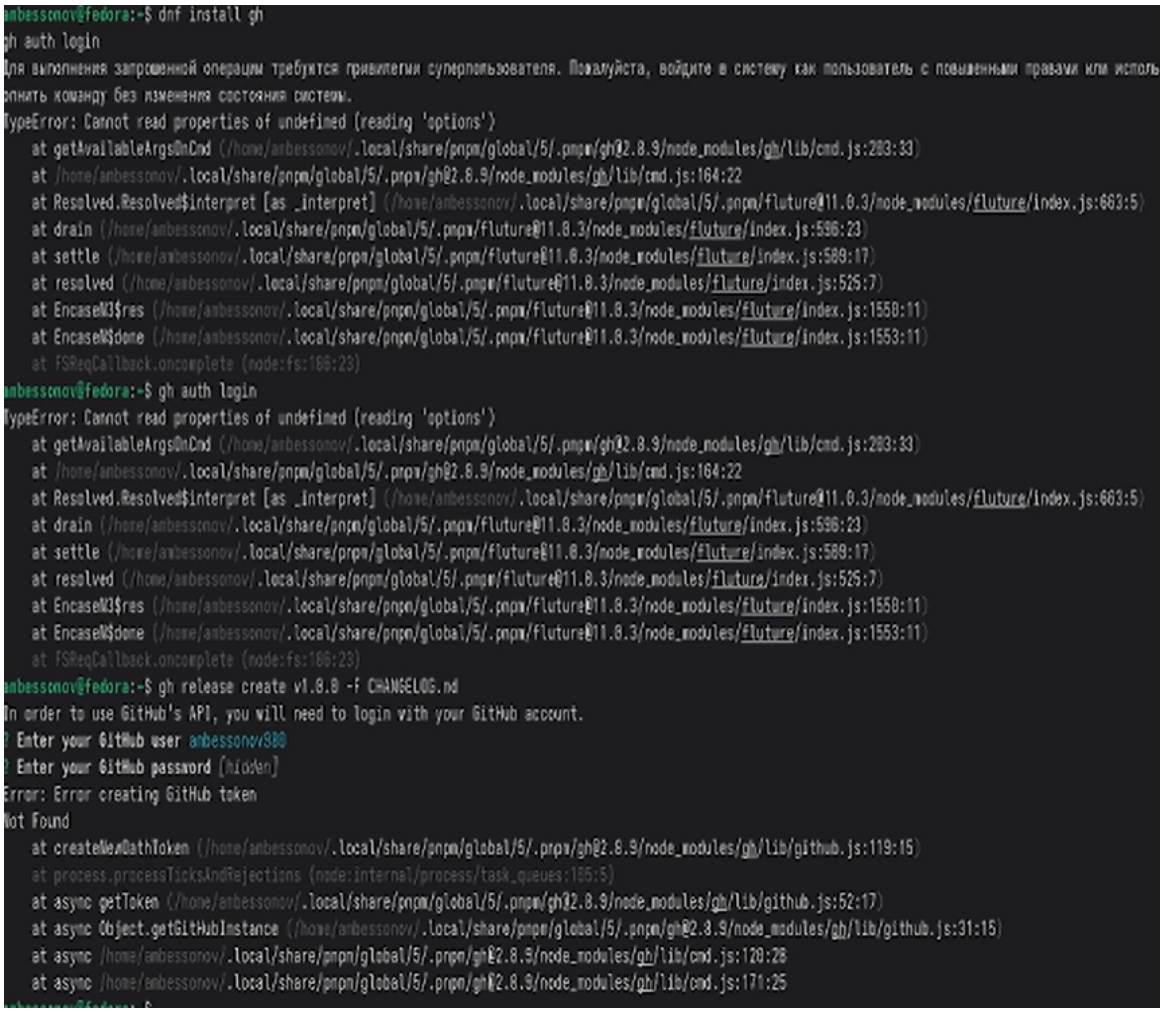

Рисунок 4.5: Рис.5

4.5 Создание релиза на GitHub

С помощью утилиты gh (GitHub CLI) создан релиз на GitHub с описанием из CHANGELOG.md. Предварительно gh был установлен и выполнена аутентификация: - sudo dnf install gh - gh auth login - gh release create v1.0.0 -F CHANGELOG.md

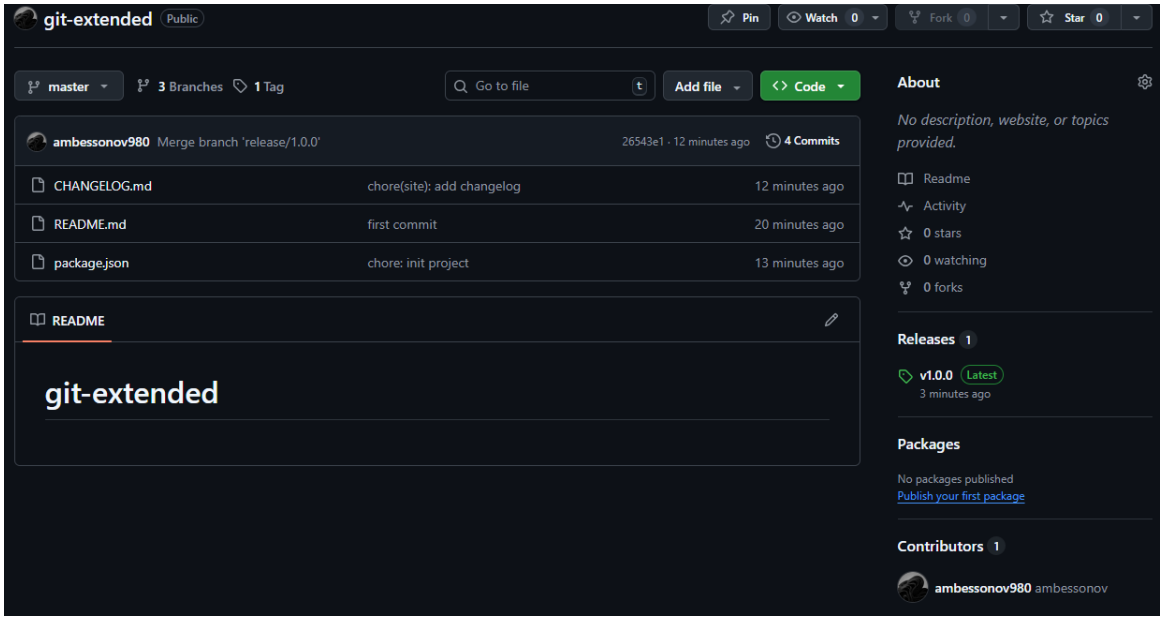

Рисунок 4.6: Рис.6

Разработка новой функциональности Для демонстрации рабочего процесса была создана функциональная ветка feature/awesome-feature: - git flow feature start awesome-feature В этой ветке был создан файл feature.js с простым кодом, затем изменения закоммичены с использованием git cz (тип feat): - echo «console.log(„Awesome feature“);» > feature.js - git add feature.js - git cz После завершения разработки ветка была влита в develop: - git flow feature finish awesome-feature

4.6 Создание релиза v1.2.3

Предположим, что после добавления нескольких функций решено выпустить новую минорную версию 1.2.3. - git flow release start 1.2.3 В файле package.json версия изменена на 1.2.3. Обновлён журнал изменений: - standard-changelog - git add CHANGELOG.md package.json - git commit -am „chore(site): update changelog for 1.2.3“ Релиз завершён: - git flow release finish 1.2.3 Изменения отправлены на GitHub: - git push –all - git push –tags И создан релиз на GitHub: - gh release create v1.2.3 -F CHANGELOG.md

4.7 Выполнение задания

```
ambessonov@fedora:~/work/study/2026-2027/Операционные системы/os-intro$ git flow init

Which branch should be used for bringing forth production releases?
   - master
Branch name for production releases: [master]
Branch name for "next release" development: [develop]

How to name your supporting branch prefixes?
Feature branches? [feature/]
Bugfix branches? [bugfix/]
Release branches? [release/]
Hotfix branches? [hotfix/]
Support branches? [support/]
Version tag prefix? [] v
Hooks and filters directory? [/home/ambessonov/work/study/2026-2027/Операционные системы/os-intro/.git/hooks]
ambessonov@fedora:~/work/study/2026-2027/Операционные системы/os-intro$ git checkout develop
git push --all
git branch --set-upstream-to=origin/develop develop
Уже на «develop»
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'develop' on GitHub by visiting:
remote:      https://github.com/ambessonov980/study_2022-2023_os-intro/pull/new/develop
remote:
To github.com:ambessonov980/study_2022-2023_os-intro.git
 * [new branch]      develop -> develop
branch 'develop' set up to track 'origin/develop'.
ambessonov@fedora:~/work/study/2026-2027/Операционные системы/os-intro$ pnpm init
Wrote to /home/ambessonov/work/study/2026-2027/Операционные системы/os-intro/package.json

{
  "name": "os-intro",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "packageManager": "pnpm@10.9.0"
}
```

Рисунок 4.7: Рис.7

```
ambessonov@fedora:~/work/study/2026-2027/Операционные системы/os-intro$ git add package.json
git cz
cz-cli@4.3.1, cz-conventional-changelog@3.3.0

? Select the type of change that you're committing: chore:    Other changes that don't modify src or test files
? What is the scope of this change (e.g. component or file name): (press enter to skip)
? Write a short, imperative tense description of the change (max 93 chars):
 (12) init project
? Provide a longer description of the change: (press enter to skip)

? Are there any breaking changes? No
? Does this change affect any open issues? No
error: gpg failed to sign the data:
[GNUPG:] KEY_CONSIDERED B5413DF0EC26894C1BCA1E49CB007C81B2651359 2
[GNUPG:] BEGIN_SIGNING H10
[GNUPG:] PINENTRY_LAUNCHED 20318 gnome3 1.3.1-unknown /dev/pts/1 xterm-256color :0 20620/1000/5 1000/1000 0
gpg: подписать не удалось: Фраза-пароль не задана
[GNUPG:] FAILURE sign 67109041
gpg: signing failed: Фраза-пароль не задана

fatal: сбой записи объекта коммита

            Git exited with code 128. Did you forget to run:

              git config --global user.email "you@example.com"
              git config --global user.name "Your Name"

git exited with error code 128
ambessonov@fedora:~/work/study/2026-2027/Операционные системы/os-intro$ git add package.json
git cz
cz-cli@4.3.1, cz-conventional-changelog@3.3.0

? Select the type of change that you're committing: chore:    Other changes that don't modify src or test files
? What is the scope of this change (e.g. component or file name): (press enter to skip)
? Write a short, imperative tense description of the change (max 93 chars):
 (12) init project
? Provide a longer description of the change: (press enter to skip)

? Are there any breaking changes? Yes
? A BREAKING CHANGE commit requires a body. Please enter a longer description of the commit itself:
 -
? Describe the breaking changes:

? Does this change affect any open issues? No
```

Рисунок 4.8: Рис.8

```
ambessonov@fedora:~/work/study/2026-2027/Операционные системы/os-intro$ git push
Перечисление объектов: 4, готово.
Подсчет объектов: 100% (4/4), готово.
При сжатии изменений используется до 16 потоков
Сжатие объектов: 100% (3/3), готово.
Запись объектов: 100% (3/3), 1.20 КиБ | 1.20 МиБ/с, готово.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To github.com:ambessonov980/study_2022-2023_os-intro.git
   447332d..4b82198  develop -> develop
ambessonov@fedora:~/work/study/2026-2027/Операционные системы/os-intro$ git flow release start 1.0.0
Переключились на новую ветку «release/1.0.0»

Summary of actions:
- A new branch 'release/1.0.0' was created, based on 'develop'
- You are now on branch 'release/1.0.0'

Follow-up actions:
- Bump the version number now!
- Start committing last-minute fixes in preparing your release
- When done, run:

     git flow release finish '1.0.0'

ambessonov@fedora:~/work/study/2026-2027/Операционные системы/os-intro$ ^C
ambessonov@fedora:~/work/study/2026-2027/Операционные системы/os-intro$ standard-changelog --first-release
ambessonov@fedora:~/work/study/2026-2027/Операционные системы/os-intro$ git add CHANGELOG.md
git commit -am 'chore(site): add changelog for v1.0.0'
[release/1.0.0 ef1895d] chore(site): add changelog for v1.0.0
 1 file changed, 5 insertions(+), 250 deletions(-)
ambessonov@fedora:~/work/study/2026-2027/Операционные системы/os-intro$ git flow release finish 1.0.0
Переключились на ветку «master»
Эта ветка соответствует «origin/master».
Merge made by the 'ort' strategy.
```

Рисунок 4.9: Рис.9

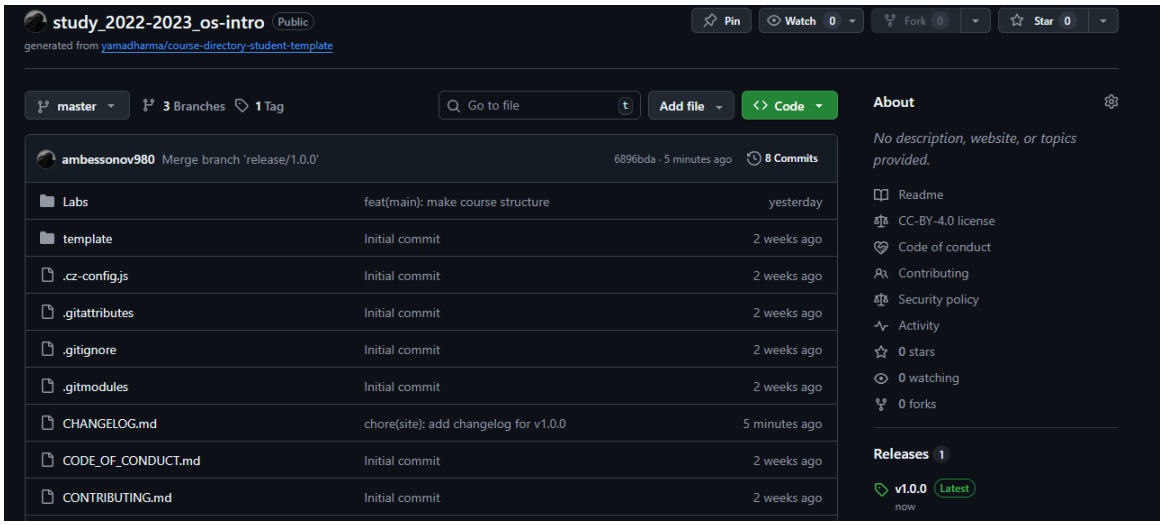

Рисунок 4.10: Рис.10

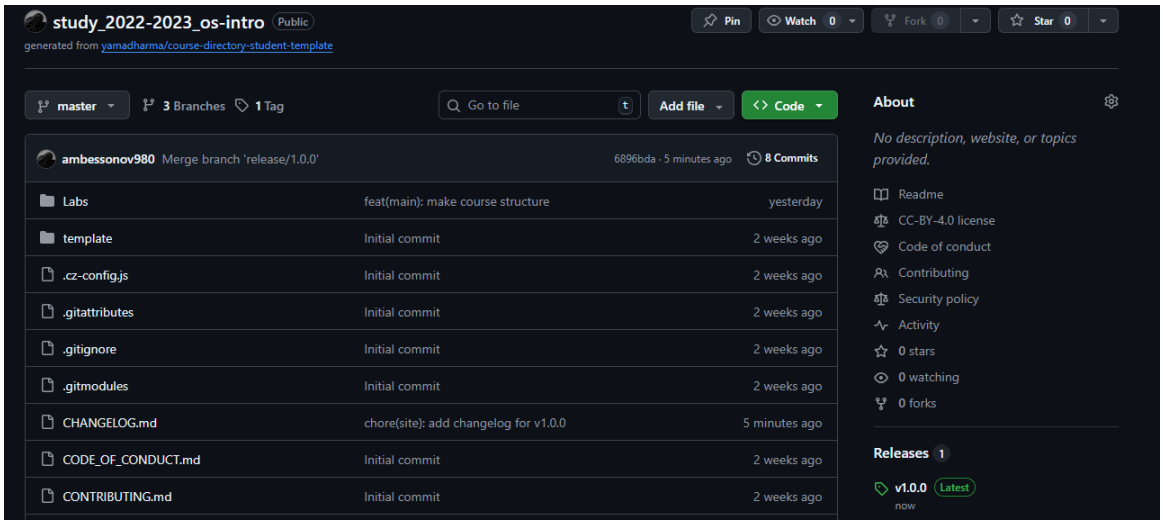

Рисунок 4.11: Рис.11

5 Выводы

В ходе лабораторной работы были изучены и применены на практике: - модель ветвления Gitflow и её реализация с помощью пакета git-flow; - семантическое версионирование (SemVer); - стандартизация коммитов согласно спецификации Conventional Commits; - использование инструментов commitizen и standard-changelog для автоматизации подготовки коммитов и журналов изменений; - создание релизов на GitHub с помощью утилиты gh. - Полученные навыки позволяют организовать эффективную командную работу над проектами с чёткой историей изменений и автоматическим управлением версиями.

Список литературы